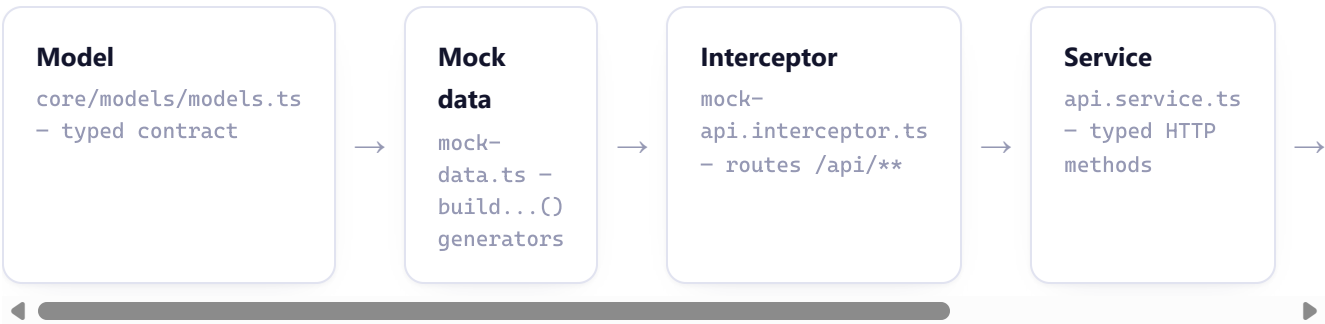


Component Documentation

Every FAM screen is real UI wired to deterministic mock data behind one interceptor — so it runs and demos today, and only the data source changes when a live API lands.

01 Architecture at a glance

A component never touches `HttpClient` or mock data directly — it only reads a `Store`. That single seam is what makes "component + mock data" a clean, swappable delivery.



Only the first three boxes change when a real backend replaces the mock — Service, Store and Component are untouched.

LAYER	FILE(S)	RESPONSIBILITY
Models	core/models/models.ts	Typed response/row shapes — contract between mock and real API
Mock data	core/mock-api/mock-data.ts	build...() generators producing deterministic fake payloads
Mock routing	core/mock-api/mock-api.interceptor.ts	Matches /api/** paths to a generator, adds artificial latency
Service	core/services/api.service.ts	One typed method per endpoint, returns Observable<T>
Store	core/state/*.store.ts	Signal-based state: loading/error/data, caching, pagination
Component	features/**	Template + computed() widget configs — no fetching logic

02 Shared UI atoms

`shared/components/ui.components.ts` — small building blocks used across every screen.

SELECTOR	PURPOSE	KEY INPUTS
<code><fam-kpi></code>	Single KPI tile — label / value / unit / sub, optional accent color	label*, value*, unit, sub, accent
<code><fam-loading></code>	Inline spinner + "Loading {what}" while a store query is in flight	what
<code><fam-state-legend></code>	Fixed legend chips for the tool states shown below	withGap
<code><fam-trend></code>	▲ / ▼ percentage pill; color flips via badWhenUp	value*, badWhenUp

Tool state legend

● Production ● Engineering ● Standby ● Scheduled Downtime ● Unscheduled Downtime ● Gap

Alarm category palette

● Equipment Safety ● Attention Flags ● Data Integrity ● Irrecoverable

03 Dynamic widget system

`shared/dynamic/*` — every feature screen renders a page as **data**, not template. A component computes a `WidgetConfig[]` and hands it to `<fam-dynamic-page>`. Adding a panel means adding an object to that array, not writing new HTML.

TYPE	INTERFACE	RENDERS	USED FOR
<code>kpi-grid</code>	<code>KpiGridWidget</code>	Auto-fit grid of <code><fam-kpi></code> tiles	KPI rows
<code>chart</code>	<code>ChartWidget</code>	Chart.js canvas, optional <code>onPointClick</code> drill-down	Trends, bar / line / pareto charts
<code>table</code>	<code>TableWidget<Row></code>	text/mono/badge/dot/trend cells, grouping, pagination	Breakdown / summary / event tables
<code>ranked-list</code>	<code>RankedListWidget</code>	Rank + title + value + trend pill + progress bar	"Top 10" drill-down lists

TYPE	INTERFACE	RENDERS	USED FOR
<code>component</code>	<code>ComponentWidget</code>	Arbitrary component, direct class or name + registry	Embedding bespoke panels in a dynamic page

Widget registry — `widget-registry.ts` / `register-widgets.ts`

let a `component`-type widget reference a component *by string name* instead of an import, so dynamic pages can embed feature-specific widgets like `state-heatmap` or `alarm-info-panel` .

04 Store foundation

`core/state/base.store.ts` — every feature store is composed from three primitives instead of hand-written loading/error booleans.

PRIMITIVE	GIVES YOU
<code>createQuery(fetcher, opts)</code>	status (idle/loading/loaded/error), TTL cache, in-flight de-dupe, request cancellation, refresh(), clear()
<code>createPagination(source, size)</code>	Client-side paging over any <code>Signal<T[]></code> — page, pageCount, paged, next/prev/goTo
<code>createListFilter(source, initial, fn)</code>	Reactive predicate filtering over a <code>Signal<T[]></code>

Existing stores built this way: `UptimeStore` `AlarmStore` `FilterStore` .

05 Auth

LoginComponent `/login`

Reactive form via `AuthService.login()` ; demo credentials live in `AUTH_CONFIG` . Selector: `fam-login` .

06 Fleet Up+Time Analysis

Route `/fleet-availability/up-time/analysis` · endpoints `GET /api/uptime/analysis` , `GET /api/uptime/trend`

UptimeAnalysisComponent `fam-uptime-analysis`

Reads `UptimeStore.analysis()` + `.trend()` . Trend chart (1W/13W rolling + period average), rolling/tool/SW-version breakdown table, top-10 unavailable tools chart, downtime category table. CSV export.

07 **Fleet Up+Time Availability**

Route `/fleet-availability/up-time/availability` with child tabs `heatmap` / `gantt` / `events` / `activities` · endpoints `GET /api/uptime/availability` , `GET /api/tools/state-segments` , `GET /api/tools/segment-activities`

COMPONENT	SELECTOR	READS	PURPOSE
UptimeAvailabilityComponent	<code>fam-uptime-availability</code>	<code>.availability()</code>	Page shell — KPI row, trend + top-unavailable widgets, tool selector, routed tabs, downtime table
StateHeatmapComponent	<code>fam-state-heatmap</code>	<code>.availability()?.heatmap</code>	24h × 14-day state heatmap <code>state-heatmap</code>
ActivityGanttComponent	<code>fam-activity-gantt</code>	<code>.gantt / .ganttSummary</code>	Per-day Sys/Tool gantt bars, day-shift mask, paging <code>activity-gantt</code>
EventDetailsComponent	<code>fam-event-details</code>	<code>.events() / .pagedEvents()</code>	State-change events, grouped by day, paginated <code>event-details</code>
SegmentActivitiesComponent	<code>fam-segment-activities</code>	<code>.segmentActivities()</code>	Task/recipe-level activity table correlated to production windows

Heatmap, Gantt and Event Details are all derived from the **same underlying StateSegment feed** (`segment-derivation.util.ts`), so the three tabs never disagree with each other.

08 **Alarm Explorer**

Data source: `AlarmStore` throughout · endpoints under `GET /api/alarms/**`

COMPONENT	ROUTE	PURPOSE
AlarmHomeComponent	<code>/alarm-explorer</code>	Alarm volume chart (trend/pareto toggle), fleet breakdown → drill into a fleet
FleetDetailComponent	<code>/alarm-explorer/fleet/:fleetId</code>	KPIs, tool-level distribution chart, top alarms, tool summary → drill into a tool
ToolAlarmsComponent	<code>.../tool/:toolId</code>	Searchable/filterable alarm table + info side panel → drill into an alarm
AlarmEventsComponent	<code>.../alarm/:alarmId</code>	Chronological event log, recipe filter, pagination, CSV export
AlarmInfoPanelComponent	<code>alarm-info-panel</code>	Inspector panel for the alarm selected in ToolAlarmsComponent's table

09 Not yet built

Served today by `PlaceholderComponent` as "scaffolded, coming soon."

These are the next candidates for component-by-component delivery.

ROUTE	MODULE
<code>/fleet-configuration</code>	Fleet Configuration
<code>/fleet-productivity</code>	Fleet Productivity
<code>/tqual</code>	TQual
<code>/my-reports</code>	My Reports
<code>/innovation-lab</code>	Innovation Lab
<code>/engineering-utilities</code>	Engineering Utilities

10 Delivering a new component

The repeatable sequence for turning one placeholder module — or any new screen — into a real, demoable component backed by mock data. Each step depends on the one before it.

1

Model

Add response/row interfaces to `core/models/models.ts`.

2

Mock data

Add a `build...()` generator to `core/mock-api/mock-data.ts`.

3

Mock route

Add the matching path branch in `mock-api.interceptor.ts`.

4

Service

Add a typed method to `ApiService` (`core/services/api.service.ts`).

5

Store

Add or extend a signal Store under `core/state/`, composing `createQuery` (+ `createPagination` / `createListFilter` as needed).

6

Component

Build the screen, preferring `WidgetConfig` / `<fam-dynamic-page>` composition over bespoke templates.

7

Route

Replace the placeholder route entry in `app.routes.ts` with the real `loadComponent`.

This keeps every delivered component demoable standalone — no backend required — and turns the eventual real-API integration into a one-file swap (the interceptor), not a component rewrite.

11 Model reference

Full definitions in `core/models/models.ts`, grouped by feature area.

AREA	TYPES
Global	<code>GlobalFilters</code> , <code>ToolState</code> , <code>AlarmCategory</code> , <code>Severity</code>
Up+Time Analysis	<code>WeeklyUptimePoint</code> , <code>UptimeBreakdownRow</code> , <code>UnavailableTool</code> , <code>DowntimeCategory</code> , <code>UptimeAnalysisResponse</code> , <code>UptimeTrendResponse</code>

AREA	TYPES
Up+Time Availability	AvailabilityKpis, FleetTrendPoint, HeatmapDay, StateTotals, GanttSegment, GanttDay, ToolEvent, AvailabilityResponse, StateSegment, SegmentActivity
Alarm Explorer	FleetAlarmSummary, AlarmVolumeWeek, ToolAlarmSummary, AlarmDefinition, AlarmEvent, AlarmHomeResponse, FleetAlarmDetailResponse, ToolAlarmDetailResponse, AlarmEventsResponse